Analysis of Agentic Typestate Specification Synthesis

Workshop Participant

ABSTRACT

We analyze 168 successful runs of Barista, an AI agent system that synthesizes typestate specifications for Java standard library classes. Across 35 classes at four complexity levels, we find that specification complexity significantly affects agent cost ($\epsilon^2 = 0.24$, $p < 10^{-8}$), duration ($\epsilon^2 = 0.24$, $p < 10^{-8}$), and MCP tool usage ($\epsilon^2 = 0.15$, $p < 10^{-5}$). Complex classes cost roughly 3× more than classes with no typestate protocol. Intraclass correlation analysis shows excellent reliability (ICC = 0.86 for cost), indicating that agent effort is primarily determined by the target class rather than run-to-run variation.

ACM Reference Format:
Workshop Participant. 2026. Analysis of Agentic Typestate Specification Synthesis. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Typestate protocols specify which methods may be called on an object depending on its current state. For example, a `Socket` must be connected before data can be sent, and cannot be used after closing. Encoding these protocols as formal specifications enables static verification of correct API usage, but writing such specifications manually is labor-intensive.

Barista is a system that uses AI agents, powered by large language models, to automatically synthesize typestate specifications for Java classes. The agent analyzes class documentation and source code, produces a state machine, and encodes it as LiquidJava annotations. This process involves multiple sub-agents (protocol analyzer, specification writer, test generator, verifier) coordinated by an orchestrator.

In this paper, we analyze the cost, duration, and tool usage patterns of Barista across 35 Java standard library classes, each processed 5 independent times. We examine how the complexity of the target specification affects agent effort and assess the reproducibility of the synthesis process.

2 METHODOLOGY

2.1 Dataset

We selected 35 classes from the Java standard library spanning packages including `java.net`, `java.io`, `java.util`, `javax.management`, and `javax.security`. Each class was independently processed by Barista 5 times, yielding 175 total runs. After filtering 7 runs with zero cost and zero SDK turns (likely failed executions), 168 valid runs remain.

Table 1: Agent metrics by specification complexity. Values are mean ± SD.

	Classes	Runs	Cost (USD)	Duration (s)	MCP Calls
None	6	30	0.94 ± 0.99	192 ± 196	2.8 ± 3.6
Simple	9	45	2.05 ± 1.21	371 ± 184	4.2 ± 2.0
Moderate	7	29	2.56 ± 1.70	466 ± 193	5.6 ± 3.2
Complex	13	64	2.93 ± 1.85	515 ± 267	5.7 ± 3.7

2.2 Complexity Classification

Each class was independently rated by two reviewers using a rubric based on structural factors: branching transitions, scale (>5 unique transition patterns or ≥20 annotated methods), multiple state sets, state-dependent value refinements, and encoding workarounds. Classes with no state protocol are rated *none*; zero factors yields *simple*; one factor yields *moderate*; two or more factors yields *complex*. The final distribution is: none (6 classes, 30 runs), simple (9 classes, 45 runs), moderate (7 classes, 29 runs), and complex (13 classes, 64 runs).

2.3 Metrics

Each run produces a workflow log recording: wall-clock duration (seconds), total API cost (USD), SDK conversation turns, per-tool call counts (Read, Edit, Bash, Write, Grep, Glob), MCP tool calls (validate_dfa, verify_liquidjava, generate_tests, generate_refinements), sub-agent spawn counts, and tool errors.

2.4 Statistical Methods

We use Kruskal-Wallis tests (non-parametric) for four-group comparisons across complexity levels, with Dunn's post-hoc test and Bonferroni correction for pairwise comparisons. Effect sizes are reported as epsilon-squared (ϵ^2). For the two-group comparison of "none" vs. all other classes, we use Mann-Whitney U with rank-biserial correlation as effect size. Spearman correlations are computed on class-level means ($n = 35$) to avoid inflated significance from non-independent within-class runs. Reproducibility is assessed via coefficient of variation (CV) and intraclass correlation coefficient ICC(1,1).

3 RESULTS

3.1 Agent Effort by Complexity

Table 1 summarizes key metrics by complexity level. Across all 168 valid runs, the median cost per run was \$1.91 (mean \$2.27, SD \$1.69) and the mean duration was 6.8 minutes. Total expenditure across all runs was \$381.

Specification complexity significantly affects agent cost (Kruskal-Wallis $H = 39.2$, $p < 10^{-8}$, $\epsilon^2 = 0.24$, large effect) and duration

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Association for Computing Machinery.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

($H = 39.8$, $p < 10^{-8}$, $\epsilon^2 = 0.24$, large). Dunn's post-hoc tests show that "none" differs significantly from all other levels ($p < 0.03$ for all pairs), while moderate and complex do not differ from each other ($p = 1.0$).

3.2 Baseline: Classes Without Typestate

Classes with no state protocol serve as a baseline for the minimum agent effort required. Their median cost ($0.26) is significantly lower than classes with a protocol (median $2.24; Mann-Whitney $p < 10^{-8}$, rank-biserial $r = 0.65$, large effect). The same pattern holds for duration (median 74s vs. 444s, $r = 0.62$) and agent turns (median 8.5 vs. 70, $r = 0.50$).

3.3 Tool Usage Patterns

Complex classes require substantially more tool calls (mean 91.5) than none-complexity classes (mean 32.2). The increase is concentrated in Read calls (41.2 vs. 16.4), Edit calls (9.1 vs. 1.2), and MCP tool calls. MCP usage differs significantly across complexity levels ($H = 24.6$, $p < 10^{-5}$, $\epsilon^2 = 0.15$, large effect), with the validate_dfa and verify_liquidjava tools called approximately twice as often for complex classes as for none classes.

Agent turns also increase with complexity ($H = 20.2$, $p < 10^{-4}$, $\epsilon^2 = 0.12$, medium effect), rising from a mean of 36.8 turns for none to 98.5 for complex classes.

3.4 Reproducibility

The median coefficient of variation for cost across the 5 runs of each class is 0.35, indicating moderate run-to-run variability. ICC(1,1) analysis on 30 classes with balanced runs shows excellent reliability: 0.86 for cost, 0.87 for duration, and 0.82 for agent turns. This confirms that the majority of variance is between-class (i.e., driven by the target class) rather than within-class noise.

3.5 Cost Drivers

At the class level ($n = 35$), cost correlates strongly with duration and total tool calls (Spearman correlations computed on class-level means to account for within-class non-independence). The most expensive class was `DefaultMutableTreeNode` (complex; $5.52 ± $2.37 per run) and the cheapest was `FlatteningPathIterator` (none; $0.17 ± $0.02).

4 DISCUSSION

5 CONCLUSION